SYSTEM ARCHITECTURE SPECIFICATION

API Data Flow & Sync Logic Design

Detailed sequence flows for hybrid food search caching, local meal persistence, Strava OAuth workout sync, and on-device lifecycle triggers.

1. Hybrid Food Search & Caching Architecture

To eliminate unnecessary API calls and ensure zero-latency offline operation, food queries query local SQLite persistence prior to issuing external network requests.

```
[User Enters Query] ──▶ [Query SQLite `foods` Table] ──(Found?)──▶ [Display Local Cached Results]
```

2. Data Flow Specifications

Process Component	Trigger & Data Source	Execution & State Outcome
Local Meal Logging	User action on Log Screen; reads from local `foods` table.	Writes row to `food_logs` with local UUID and `YYYY-MM-DD` index. Immediately updates dashboard state.
Calorie Threshold Check	Triggered inside SQLite transaction upon meal logging.	Evaluates `total_consumed > daily_target`. Instantly renders Over-Calorie Warning Modal if breached.
Strava Activity Sync	App launch or Dashboard pull-to-refresh; Strava REST API.	Fetches activities via GET `/athlete/activities`. De-duplicates by activity ID and saves to `exercise_logs`.
Barcode Scanner Lookup	Camera UPC scan on Log Screen; Open Food Facts API.	Queries Open Food Facts by barcode. Parses macros and caches entry into `foods` master table.

3. Local Notification & Lifecycle Engines

Local Reminders Engine (`expo-notifications`)

Meal reminder notifications execute 100% on-device without external server dependency. Fixed daily cron triggers schedule local notifications for Breakfast (08:00), Lunch (12:30), Dinner (18:30), and Snacks (15:00).

Weekly Weigh-In & Motivation Engine

Upon app mount, a lifecycle hook queries the maximum `logged_date` in `weight_logs`. If the delta is ≥ 7 days, a weigh-in reminder overlay is displayed. Evaluating the weight delta against historical trends generates an adaptive motivational message.